

# Java Assignment – 7

## Abstraction, Interface, Exception Handling and Collections

### 1. Abstraction – Vehicle Rental System

Problem Statement:

Create a Java program to demonstrate Abstraction using an abstract class `Vehicle`.

#### Abstract Class – `Vehicle`

Properties:

- `vehicleNo`
- `brand`
- `rentalRate`

Constructor: Initialize all the properties.

Methods:

- `displayVehicleDetails()` – Display vehicle details.
- `calculateRental(int days)` – Declare this as an abstract method.

#### Subclass – `Car`

Additional Properties:

- `numberOfSeats`
- `insuranceCharge`

Methods:

- Implement `calculateRental(int days)` to calculate the total rental amount, including the insurance charge.
- `displayCarDetails()` – Display car-specific details.

#### Requirements

- Create an abstract class `Vehicle`.
- Implement the abstract method in the `Car` class.
- Create a `Car` object and display its details and rental amount for the specified number of days.

## 2. Interface – Payment Processing System

Problem Statement:

Create a Java program to demonstrate an Interface using a payment processing system.

### Interface – `Payment`

Declare the following methods:

- `processPayment(double amount)`
- `displayPaymentDetails()`

### Class – `UPIPayment`

Properties:

- `transactionId`
- `upiId`

Implement the `Payment` interface.

Methods:

- `processPayment(double amount)` – Display the UPI payment amount and transaction details.
- `displayPaymentDetails()` – Display the payment information.

### Class – `CardPayment`

Properties:

- `transactionId`
- `cardNumber`

Implement the `Payment` interface.

Methods:

- `processPayment(double amount)` – Display the card payment amount and transaction details.
- `displayPaymentDetails()` – Display the payment information.

## Requirements

- Create the `Payment` interface.
- Implement it in both `UPIPayment` and `CardPayment`.
- Create objects of both classes.
- Process payments using both payment methods and display their details.

### 3. Exception Handling – Electricity Bill Processing

Problem Statement:

Create a Java program to demonstrate exception handling using `try`, `catch`, and `finally` while processing an electricity bill.

#### Class – `ElectricityBill`

Properties:

- `consumerNo`
- `consumerName`
- `unitsConsumed`

Constructor: Initialize the consumer details.

Methods:

- `readDetails()` – Read consumer details and units consumed.
- `calculateBill()` – Calculate the bill based on the units consumed.
- `displayBill()` – Display the consumer details and bill amount.

#### Requirements

- Read the units consumed from the user.
- Calculate the bill amount using the following rates:
  - Up to 100 units: ₹3 per unit.
  - 101–200 units: ₹5 per unit.
  - Above 200 units: ₹8 per unit.
- Use a `try` block to read and process the input.
- Use a `catch` block to handle invalid numeric input using `InputMismatchException`.
- Use a `finally` block to display the message "Electricity bill processing completed."
- Ensure that negative units are rejected using appropriate validation.

### 4. Collection – ArrayList of Product Objects

Problem Statement:

Create a Java program to demonstrate the use of an `ArrayList` to store and display Product objects.

#### Class – `Product`

Properties:

- `productId`
- `productName`

- `price`

Constructor: Initialize all the properties.

Methods:

- `read()` – Read product details.
- `display()` – Display product details.

## Requirements

- Create an `ArrayList<Product>`.
- Read details of five products from the user.
- Create a `Product` object for each product and add it to the `ArrayList`.
- Display all product details.
- Calculate and display the total price of all products.
- Display the details of the product with the highest price.

Restriction: Use an `ArrayList` of `Product` objects.